Actividad: Estructuras y paso por referencia

Programación I, Semana 9

Nombres: _____ y _____

Sección: _____

Esta actividad **no tiene nota**.

Datos para todos los ejercicios: un equipo de fútbol.

```
struct Jugador {
    std::string nombre;
    int goles;
    int partidos;
};

const int N = 4;
Jugador equipo[N] = {
    {"Leo", 7, 10},     // 0
    {"Ana", 8, 6},      // 1
    {"Juan", 2, 7},     // 2
    {"Sara", 9, 12}     // 3
};
```

Ejercicios obligatorios

Ejercicio 1: El goleador

Completar los espacios para que la función devuelva la **posición** del jugador con más goles.

Pista: es el máximo de la clase, aplicado a un campo. Se guarda la posición, no la cantidad de goles.

```
int posicionGoleador(Jugador e[], int n) {
    int mejor = ______;
    for (int i = 1; i < n; i++) {
        if (________________ > ________________) {
            mejor = ______;
        }
    }
    return mejor;
}
```

Resultado esperado: `posicionGoleador(equipo, N)` devuelve **3**.

Escribir el código de `main` que imprime el **nombre** del goleador:

Ejercicio 2: Seguimiento

```
int buscarJugador(Jugador e[], int n, std::string nombre) {
    int pos = -1;
    for (int i = 0; i < n && pos == -1; i++) {
        if (e[i].nombre == nombre) {
            pos = i;
        }
    }
    return pos;
}
```

Completar la tabla de seguimiento de `buscarJugador(equipo, N, "Juan")`.
Resultado esperado: la función devuelve **2**.

i	**e[i].nombre**	**¿== "Juan"?**	**pos**
inicio			

¿Qué devuelve `buscarJugador(equipo, N, "Pepe")`? _____

Ejercicio 3: ¿Qué imprime?

a)

```
Jugador a = {"Leo", 3, 5};
Jugador b = a;
b.goles = 10;
std::cout << a.goles << " " << b.goles;
```

Imprime: _____

b)

Pista: observar cuál de las dos funciones tiene &.

```
void sumarGol(Jugador j) {
    j.goles = j.goles + 1;
}

void sumarGolRef(Jugador& j) {
    j.goles = j.goles + 1;
}

int main() {
    Jugador p = {"Leo", 7, 10};
    sumarGol(p);
    sumarGolRef(p);
    sumarGolRef(p);
    std::cout << p.goles;
    return 0;
}
```

Imprime: _____

c)

```
void f(int a, int& b) {
    a = a + 1;
    b = b + 1;
}

int main() {
    int x = 1;
    int y = 1;
    f(x, y);
    std::cout << x << " " << y;
    return 0;
}
```

Imprime: _____

Ejercicio 4: Encontrar los errores

Este programa debería sumarle un gol a Leo e imprimir sus goles, pero tiene **4 errores**. Encerrar cada uno en un círculo y escribir al lado la corrección.

Resultado esperado: ya corregido, el programa imprime **8**.

```
struct Jugador {
    std::string nombre;
    int goles;
    int partidos;
}

void sumarGol(Jugador j) {
    j.goles = j.goles + 1;
}

int main() {
    Jugador equipo[4] = {{"Leo", 7, 10}, {"Ana", 8, 6},
                         {"Juan", 2, 7}, {"Sara", 9, 12}};
    for (int i = 0; i < 4; i++) {
        if (equipo[i].nombre = "Leo") {
            sumarGol(equipo[i]);
        }
    }
    std::cout << equipo.goles[0];
    return 0;
}
```

Ejercicios intermedios

Ejercicio 5: Registrar un partido

Escribir **completas** estas dos funciones:

- `void registrarPartido(Jugador& j, int golesAnotados)`: suma 1 a los partidos del jugador y le suma los goles que anotó.
- `double promedioGoles(Jugador j)`: devuelve los goles por partido. Si el jugador no tiene partidos, devuelve 0.

Resultado esperado: después de `registrarPartido(equipo[2], 2)`, Juan tiene 4 goles en 8 partidos, y `promedioGoles(equipo[2])` devuelve **0.5**.

¿Por qué `registrarPartido` lleva `&` y `promedioGoles` no?

Reto

Ejercicio 6: El mejor promedio

Escribir la función `Jugador mejorPromedio(Jugador e[], int n)`, que devuelve el **jugador completo** (no su posición) con más goles por partido. Debe usar `promedioGoles` del ejercicio 5. Si hay empate, el primero.

> *Pista: buscar la posición como en el máximo y, al final, devolver `e[posicion]`.*

Resultado esperado: con el `equipo` original devuelve a **Ana**. ¿Por qué no es Sara, si Sara tiene más goles?